JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class AtomicLong

java.lang.Object
    java.lang.Number
        java.util.concurrent.atomic.AtomicLong

All Implemented Interfaces:

Serializable

```
public class AtomicLong
extends Number
implements Serializable
```

A `long` value that may be updated atomically. See the `VarHandle` specification for descriptions of the properties of atomic accesses. An `AtomicLong` is used in applications such as atomically incremented sequence numbers, and cannot be used as a replacement for a `Long`. However, this class does extend `Number` to allow uniform access by tools and utilities that deal with numerically-based classes.

Since:

1.5

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
`AtomicLong()`	Creates a new AtomicLong with initial value `0`.
`AtomicLong(long initialValue)`	Creates a new AtomicLong with the given initial value.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
`final long`	`accumulateAndGet(long x, LongBinaryOperator accum`	Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`)

		the current value with the results of applying the given function to the current and given values, returning the updated value.
final long	**addAndGet**(long delta)	Atomically adds the given value to the current value, with memory effects as specified by VarHandle.getAndAdd(Object...).
final long	**compareAndExchange**(long expectedValue, long newValue)	Atomically sets the value to newValue if the current value, referred to as the *witness value*, == expectedValue, with memory effects as specified by VarHandle.compareAndExchange(Object...
final long	**compareAndExchangeAcquir**(long expectedValue, long newValue)	Atomically sets the value to newValue if the current value, referred to as the *witness value*, == expectedValue, with memory effects as specified by VarHandle.compareAndExchangeAcquire(Obje
final long	**compareAndExchangeReleas**(long expectedValue, long newValue)	Atomically sets the value to newValue if the current value, referred to as the *witness value*, == expectedValue, with memory effects as specified by VarHandle.compareAndExchangeRelease(Obje

final boolean	**compareAndSet**(long expectedValue, long newValue)	Atomically sets the value to newValue if the current value == expectedValue, with memory effects as specified by VarHandle.compareAndSet(Object...).
final long	**decrementAndGet**()	Atomically decrements the current value, with memory effects as specified by VarHandle.getAndAdd(Object...).
double	**doubleValue**()	Returns the current value of this AtomicLong as a double after a widening primitive conversion, with memory effects as specified by VarHandle.getVolatile(Object...).
float	**floatValue**()	Returns the current value of this AtomicLong as a float after a widening primitive conversion, with memory effects as specified by VarHandle.getVolatile(Object...).
final long	**get**()	Returns the current value, with memory effects as specified by VarHandle.getVolatile(Object...).
final long	**getAcquire**()	Returns the current value, with memory effects as specified by VarHandle.getAcquire(Object...).
final long	**getAndAccumulate**(long x, LongBinaryOperator accum	Atomically updates (with memory effects as specified by VarHandle.compareAndSet(Object...)) the current value with the results of applying

		the given function to the current and given values, returning the previous value.
final long	**getAndAdd**(long delta)	Atomically adds the given value to the current value, with memory effects as specified by VarHandle.getAndAdd(Object...).
final long	**getAndDecrement**()	Atomically decrements the current value, with memory effects as specified by VarHandle.getAndAdd(Object...).
final long	**getAndIncrement**()	Atomically increments the current value, with memory effects as specified by VarHandle.getAndAdd(Object...).
final long	**getAndSet**(long newValue)	Atomically sets the value to newValue and returns the old value, with memory effects as specified by VarHandle.getAndSet(Object...).
final long	**getAndUpdate**(LongUnaryOperator updat	Atomically updates (with memory effects as specified by VarHandle.compareAndSet(Object...)) the current value with the results of applying the given function,

		returning the previous value.
final long	**getOpaque**()	Returns the current value, with memory effects as specified by VarHandle.getOpaque(Object...).
final long	**getPlain**()	Returns the current value, with memory semantics of reading as if the variable was declared non-volatile.
final long	**incrementAndGet**()	Atomically increments the current value, with memory effects as specified by VarHandle.getAndAdd(Object...).
int	**intValue**()	Returns the current value of this AtomicLong as an int after a narrowing primitive conversion, with memory effects as specified by VarHandle.getVolatile(Object...).
final void	**lazySet**(long newValue)	Sets the value to newValue, with memory effects as specified by VarHandle.setRelease(Object...).
long	**longValue**()	Returns the current value of this AtomicLong as a long, with memory effects as specified by VarHandle.getVolatile(Object...).

final void	**set**(long newValue)	Sets the value to newValue, with memory effects as specified by VarHandle.setVolatile(Object...).
final void	**setOpaque** (long newValue)	Sets the value to newValue, with memory effects as specified by VarHandle.setOpaque(Object...).
final void	**setPlain**(long newValue)	Sets the value to newValue, with memory semantics of setting as if the variable was declared non-volatile and non-final.
final void	**setRelease** (long newValue)	Sets the value to newValue, with memory effects as specified by VarHandle.setRelease(Object...).
String	**toString**()	Returns the String representation of the current value.
final long	**updateAndGet** (LongUnaryOperator updat	Atomically updates (with memory effects as specified by VarHandle.compareAndSet(Object...)) the current value with the results of applying the given function, returning the updated value.
final boolean	**weakCompareAndSet** (long expectedValue, long newValue)	**Deprecated.** This method has plain memory effects but the method name implies volatile memory effects (see methods such as compareAndExchange(long, long) and

		compareAndSet(long, long)).
final boolean	**weakCompareAndSetAcquire**(long expectedValue, long newValue)	Possibly atomically sets the value to newValue if the current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetAcquire(Objec
final boolean	**weakCompareAndSetPlain**(long expectedValue, long newValue)	Possibly atomically sets the value to newValue if the current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetPlain(Object.
final boolean	**weakCompareAndSetRelease**(long expectedValue, long newValue)	Possibly atomically sets the value to newValue if the current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetRelease(Objec
final boolean	**weakCompareAndSetVolatil**(long expectedValue, long newValue)	Possibly atomically sets the value to newValue if the current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSet(Object...).

Methods declared in class Number

byteValue, shortValue

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Details

AtomicLong

```
public AtomicLong(long initialValue)
```

Creates a new AtomicLong with the given initial value.

Parameters:

initialValue - the initial value

AtomicLong

```
public AtomicLong()
```

Creates a new AtomicLong with initial value 0.

Method Details

get

```
public final long get()
```

Returns the current value, with memory effects as specified by VarHandle.getVolatile(Object...).

Returns:

the current value

set

```
public final void set(long newValue)
```

Sets the value to newValue, with memory effects as specified by VarHandle.setVolatile(Object...).

Parameters:

newValue - the new value

lazySet

```
public final void lazySet(long newValue)
```

Sets the value to `newValue`, with memory effects as specified by `VarHandle.setRelease(Object...)`.

Parameters:

`newValue` - the new value

Since:

1.6

getAndSet

```
public final long getAndSet(long newValue)
```

Atomically sets the value to `newValue` and returns the old value, with memory effects as specified by `VarHandle.getAndSet(Object...)`.

Parameters:

`newValue` - the new value

Returns:

the previous value

compareAndSet

```
public final boolean compareAndSet(long expectedValue,
                                   long newValue)
```

Atomically sets the value to `newValue` if the current value `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndSet(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful. False return indicates that the actual value was not equal to the expected value.

weakCompareAndSet

```
@Deprecated(since="9")
public final boolean weakCompareAndSet(long expectedValue,
                                       long newValue)
```

Deprecated.
This method has plain memory effects but the method name implies volatile memory effects (see methods such as `compareAndExchange(long, long)` and `compareAndSet(long, long)`). To avoid confusion over plain or volatile memory effects it is recommended that the method `weakCompareAndSetPlain(long, long)` be used instead.

Possibly atomically sets the value to `newValue` if the current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetPlain(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

See Also:

`weakCompareAndSetPlain(long, long)`

weakCompareAndSetPlain

```
public final boolean weakCompareAndSetPlain(long expectedValue,
                                            long newValue)
```

Possibly atomically sets the value to `newValue` if the current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetPlain(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

getAndIncrement

```
public final long getAndIncrement()
```

Atomically increments the current value, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Equivalent to `getAndAdd(1)`.

Returns:

the previous value

getAndDecrement

```
public final long getAndDecrement()
```

Atomically decrements the current value, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Equivalent to `getAndAdd(-1)`.

Returns:

the previous value

getAndAdd

```
public final long getAndAdd(long delta)
```

Atomically adds the given value to the current value, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Parameters:

`delta` - the value to add

Returns:

the previous value

incrementAndGet

```
public final long incrementAndGet()
```

Atomically increments the current value, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Equivalent to `addAndGet(1)`.

Returns:

the updated value

decrementAndGet

```
public final long decrementAndGet()
```

Atomically decrements the current value, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Equivalent to `addAndGet(-1)`.

Returns:

the updated value

addAndGet

```
public final long addAndGet(long delta)
```

Atomically adds the given value to the current value, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Parameters:

`delta` - the value to add

Returns:

the updated value

getAndUpdate

```
public final long getAndUpdate(LongUnaryOperator updateFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the current value with the results of applying the given function, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`updateFunction` - a side-effect-free function

Returns:

the previous value

Since:

1.8

updateAndGet

```
public final long updateAndGet(LongUnaryOperator updateFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the current value with the results of applying the given function, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`updateFunction` - a side-effect-free function

Returns:

the updated value

Since:

1.8

getAndAccumulate

```
public final long getAndAccumulate
(long x,
 LongBinaryOperator accumulatorFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the current value with the results of applying the given function to the current and given values, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads. The function is applied with the current value as its first argument, and the given update as the second argument.

Parameters:

`x` - the update value

`accumulatorFunction` - a side-effect-free function of two arguments

Returns:

the previous value

Since:

1.8

accumulateAndGet

```
public final long accumulateAndGet
(long x,
 LongBinaryOperator accumulatorFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the current value with the results of applying the given function to the current and given values, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads. The function is applied with the current value as its first argument, and the given update as the second argument.

Parameters:

`x` - the update value

`accumulatorFunction` - a side-effect-free function of two arguments

Returns:

the updated value

Since:

1.8

toString

```
public String toString()
```

Returns the String representation of the current value.

Overrides:

`toString` in class `Object`

Returns:

the String representation of the current value

intValue

```
public int intValue()
```

Returns the current value of this `AtomicLong` as an `int` after a narrowing primitive conversion, with memory effects as specified by `VarHandle.getVolatile(Object...)`.

Specified by:

`intValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `int`.

See *Java Language Specification*:

5.1.3 Narrowing Primitive Conversion

longValue

```
public long longValue()
```

Returns the current value of this `AtomicLong` as a `long`, with memory effects as specified by `VarHandle.getVolatile(Object...)`. Equivalent to `get()`.

Specified by:

`longValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `long`.

floatValue

```
public float floatValue()
```

Returns the current value of this `AtomicLong` as a `float` after a widening primitive conversion, with memory effects as specified by `VarHandle.getVolatile(Object...)`.

Specified by:

`floatValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `float`.

See *Java Language Specification*:

5.1.2 Widening Primitive Conversion

doubleValue

```
public double doubleValue()
```

Returns the current value of this `AtomicLong` as a `double` after a widening primitive conversion, with memory effects as specified by `VarHandle.getVolatile(Object...)`.

Specified by:

`doubleValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type `double`.

See *Java Language Specification*:

5.1.2 Widening Primitive Conversion

getPlain

```
public final long getPlain()
```

Returns the current value, with memory semantics of reading as if the variable was declared non-`volatile`.

Returns:

the value

Since:

9

setPlain

```
public final void setPlain(long newValue)
```

Sets the value to `newValue`, with memory semantics of setting as if the variable was declared non-`volatile` and non-`final`.

Parameters:

`newValue` - the new value

Since:

9

getOpaque

```
public final long getOpaque()
```

Returns the current value, with memory effects as specified by `VarHandle.getOpaque(Object...)`.

Returns:

the value

Since:

9

setOpaque

```
public final void setOpaque(long newValue)
```

Sets the value to `newValue`, with memory effects as specified by `VarHandle.setOpaque(Object...)`.

Parameters:

`newValue` - the new value

Since:

9

getAcquire

```
public final long getAcquire()
```

Returns the current value, with memory effects as specified by `VarHandle.getAcquire(Object...)`.

Returns:

the value

Since:

9

setRelease

```
public final void setRelease(long newValue)
```

Sets the value to `newValue`, with memory effects as specified by `VarHandle.setRelease(Object...)`.

Parameters:

`newValue` - the new value

Since:

9

compareAndExchange

```
public final long compareAndExchange(long expectedValue,
                                      long newValue)
```

Atomically sets the value to `newValue` if the current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchange(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

compareAndExchangeAcquire

```
public final long compareAndExchangeAcquire(long expectedValue,
                                             long newValue)
```

Atomically sets the value to `newValue` if the current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchangeAcquire(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

compareAndExchangeRelease

```
public final long compareAndExchangeRelease(long expectedValue,
                                             long newValue)
```

Atomically sets the value to `newValue` if the current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchangeRelease(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

weakCompareAndSetVolatile

```
public final boolean weakCompareAndSetVolatile(long expectedValue,
                                               long newValue)
```

Possibly atomically sets the value to `newValue` if the current value `==` `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSet(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

weakCompareAndSetAcquire

```
public final boolean weakCompareAndSetAcquire(long expectedValue,
                                              long newValue)
```

Possibly atomically sets the value to `newValue` if the current value `==` `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetAcquire(Object...)`.

Parameters:

expectedValue - the expected value

newValue - the new value

Returns:

true if successful

Since:

9

weakCompareAndSetRelease

```
public final boolean weakCompareAndSetRelease(long expectedValue,
                                              long newValue)
```

Possibly atomically sets the value to newValue if the current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetRelease(Object...).

Parameters:

expectedValue - the expected value

newValue - the new value

Returns:

true if successful

Since:

9

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.